Numerical Linear Algebra for Computational Sciences and Information Engineering
CITHN2006

Orthogonalization

by Nicolas Venkovic

Chair of Computational Mathematics
School of Computation, Information and Technology (CIT)
Technical University of Munich, Germany

1. *Write a function called* ***conditioned_matrix*** *that creates a random $m \times n$ matrix with condition number κ and maximum singular value $\sigma_1 = 1$.*

[1]:
```
using LinearAlgebra, Printf, BenchmarkTools, Random, SparseArrays
```

[2]:
```
function conditioned_matrix(m::Int, n::Int, cond::Float64)
  U, _ = qr(randn(m, m))
  V, _ = qr(randn(n, n))
  s = range(1.0, 1/cond; length=n)
  Σ = Diagonal(s)
  A = U[:, 1:n] * Σ * V'
  return A
end;
```

2. *Implement and test and time your implementation of Householder QR.*

[3]:
```
function HouseholderQR(A)
  m, n = size(A)
  Q = Matrix{Float64}(I, m, m)
  R = copy(A)
  for k = 1:n
    # Compute Householder vector
    v = R[k:m, k]
    v[1] -= sign(v[1]) * norm(v) # usage of sign(v[1]) allows to prevent␣
 ↪catastrophic cancellation
    beta = 2. / v'v
    # Apply Householder reflection
    R[k:m, k:n] -= beta .* v * (v'R[k:m, k:n])
    Q[1:m, k:m] -= Q[1:m, k:m] * v * (beta .* v')
  end
```

```
  return Q, R
end;
```

[4]:
```
m = 500;
n = 200;
Random.seed!(3);
A1 = conditioned_matrix(m, n, 1e10);
A2 = conditioned_matrix(m, n, 1e12);
```

[5]:
```
Q, R = @btime HouseholderQR(A1);
@printf "||A - QR|| = %.2E, ||I - Q'Q|| = %.2E" norm(A1 - Q * R) norm(I - Q'Q)
```

```
  575.413 ms (7807 allocations: 1.46 GiB)
||A - QR|| = 9.21E-15, ||I - Q'Q|| = 3.18E-14
```

[6]:
```
Q, R = @btime HouseholderQR(A2);
@printf "||A - QR|| = %.2E, ||I - Q'Q|| = %.2E" norm(A2 - Q * R) norm(I - Q'Q)
```

```
  526.294 ms (7807 allocations: 1.46 GiB)
||A - QR|| = 9.42E-15, ||I - Q'Q|| = 3.14E-14
```

3. *Implement CholeskyQR, CholeskyQR2 and Shifted CholeskyQR3.*

[7]:
```
function CholeskyQR(A)
  X = A'A # ==> if A is 1e12 then A'A ==> Ae24
  chol = cholesky(X)
  Q = A / chol.U
  return Q, chol.U
end;
```

[8]:
```
function CholeskyQR2(A)
  Q1, R1 = CholeskyQR(A)
  Q, R2 = CholeskyQR(Q1)
  return Q, R2 * R1
end;
```

[9]:
```
function ShiftedCholeskyQR(A)
  X = A'A
  s = 11 * (m * n + n * (n + 1)) * eps() * norm(A)
  X += s * I
  chol = cholesky(X)
  Q = A / chol.U
  return Q, chol.U
end;
```

[10]:
```
function ShiftedCholeskyQR3(A)
  Q1, R1 = ShiftedCholeskyQR(A)
  Q, R2 = CholeskyQR2(Q1)
  return Q, R2 * R1
```

```
end;
```

4. *Compare the runtime, loss of orthogonality and residual obtained with each CholeskyQR implementation for multiple tall-and-skinny matrices with different conditioning numbers.*

[11]:
```
try
  Q, R = @btime CholeskyQR(A1);
  @printf "||A - QR|| = %.2E, ||I - Q'Q|| = %.2E" norm(A1 - Q * R) norm(I - Q'Q)
catch e
  println("Error", e)
end
```

```
ErrorPosDefException(200)
```

[12]:
```
try
  Q, R = @btime CholeskyQR(A2);
  @printf "||A - QR|| = %.2E, ||I - Q'Q|| = %.2E" norm(A1 - Q * R) norm(I - Q'Q)
catch e
  println("Error", e)
end
```

```
ErrorPosDefException(200)
```

[13]:
```
try
  Q, R = @btime CholeskyQR2(A1);
  @printf "||A - QR|| = %.2E, ||I - Q'Q|| = %.2E" norm(A1 - Q * R) norm(I -␣
 ↪Q'Q)
catch e
  println("Error", e)
end
```

```
ErrorPosDefException(200)
```

[14]:
```
try
  Q, R = @btime CholeskyQR2(A2);
  @printf "||A - QR|| = %.2E, ||I - Q'Q|| = %.2E" norm(A2 - Q * R) norm(I - Q'Q)
catch e
  println("Error", e)
end
```

```
ErrorPosDefException(200)
```

[15]:
```
Q, R = @btime ShiftedCholeskyQR3(A1);
@printf "||A - QR|| = %.2E, ||I - Q'Q|| = %.2E" norm(A1 - Q * R) norm(I - Q'Q)
```

```
  4.804 ms (51 allocations: 5.04 MiB)
||A - QR|| = 3.95E-15, ||I - Q'Q|| = 5.32E-15
```

[16]:
```
Q, R = @btime ShiftedCholeskyQR3(A2);
@printf "||A - QR|| = %.2E, ||I - Q'Q|| = %.2E" norm(A2 - Q * R) norm(I - Q'Q)
```

```
  4.469 ms (51 allocations: 5.04 MiB)
||A - QR|| = 3.95E-15, ||I - Q'Q|| = 5.07E-15
```

5. *Implement CGS, CGS2 and MGS procedures.*

[17]:
```
function CGS_naive(X::Array{Float64,2})
  n, m = size(X)
  Q = Array{Float64,2}(undef, n, m)
  R = zeros(Float64, m, m)
  R[1, 1] = norm(X[:, 1])
  Q[:, 1] = X[:, 1] ./ R[1, 1]
  for i in 2:m
    Q[:, i] = X[:, i]
    R[1:i-1, i] .= Q[:, 1:i-1]'X[:, i] # creates temporary array => very slow
    Q[:, i] -= Q[:, 1:i-1] * R[1:i-1, i] # creates temporary array => very slow
    R[i, i] = norm(Q[:, i])
    Q[:, i] ./= R[i, i]
  end
  return Q, R
end;

function CGS(X::Array{Float64,2})
  n, m = size(X)
  Q = zeros(Float64, n, m)
  R = zeros(Float64, m, m)
  r = zeros(Float64, m)
  w = zeros(Float64, n)
  R[1, 1] = norm(X[:, 1])
  Q[:, 1] = X[:, 1] ./ R[1, 1]
  for i in 2:m
    w .= X[:, i]
    r .= (w'Q)'
    r[i:m] .= 0.
    R[:, i] = r
    w -= Q * r
    R[i, i] = norm(w)
    Q[:, i] = w ./ R[i, i]
  end
  return Q, R
end;
```

[18]:
```
function MGS_naive(X::Array{Float64,2})
  n, m = size(X)
  Q = Array{Float64,2}(undef, n, m)
  R = zeros(Float64, m, m)
  R[1, 1] = norm(X[:, 1])
  Q[:, 1] = X[:, 1] ./ R[1, 1]
  for i in 2:m
```

```
    Q[:, i] = X[:, i]
    for j in 1:i-1
      R[j, i] = Q[:, j]'Q[:, i]
      Q[:, i] -= R[j, i] .* Q[:, j]
    end
    R[i, i] = norm(Q[:, i])
    Q[:, i] ./= R[i, i]
  end
  return Q, R
end;

function MGS(X::Array{Float64,2})
  n, m = size(X)
  Q = Array{Float64,2}(undef, n, m)
  q = Vector{Float64}(undef, n)
  R = zeros(Float64, m, m)
  w = Vector{Float64}(undef, n)
  R[1, 1] = norm(X[:, 1])
  Q[:, 1] = X[:, 1] ./ R[1, 1]
  for i in 2:m
    w .= X[:, i]
    for j in 1:i-1
      q .= Q[:, j]
      R[j, i] = q'w
      w -= R[j, i] .* q
    end
    R[i, i] = norm(w)
    Q[:, i] = w ./ R[i, i]
  end
  return Q, R
end;
```

[19]:
```
function CGS2_naive(X::Array{Float64,2})
  n, m = size(X)
  Q = Array{Float64,2}(undef, n, m)
  R = zeros(Float64, m, m)
  R[1, 1] = norm(X[:, 1])
  Q[:, 1] = X[:, 1] ./ R[1, 1]
  for i in 2:m
    Q[:, i] = X[:, i]
    R[1:i-1, i] .= Q[:, 1:i-1]'X[:, i] # creates temporary array => very slow
    Q[:, i] -= Q[:, 1:i-1] * R[1:i-1, i] # creates temporary array => very slow
    r = Q[:, 1:i-1]'Q[:, i] # creates temporary array => very slow
    Q[:, i] -= Q[:, 1:i-1] * r # creates temporary array => very slow
    R[i, i] = norm(Q[:, i])
    Q[:, i] ./= R[i, i]
  end
```

```
  return Q, R
end;

function CGS2(X::Array{Float64,2})
  n, m = size(X)
  Q = zeros(Float64, n, m)
  R = zeros(Float64, m, m)
  r = zeros(Float64, m)
  w = zeros(Float64, n)
  R[1, 1] = norm(X[:, 1])
  Q[:, 1] = X[:, 1] ./ R[1, 1]
  for i in 2:m
    w .= X[:, i]
    r .= Q'w
    r[i:m] .= 0.
    R[:, i] = r
    w -= Q * r
    r .= Q'w
    r[i:m] .= 0.
    w -= Q * r
    R[i, i] = norm(w)
    Q[:, i] = w ./ R[i, i]
  end
  return Q, R
end;
```

6. *Compare the runtime, loss of orthogonality and residual obtained with each Gram-Schmidt implementation for multiple tall-and-skinny matrices with different conditioning numbers.*

[20]:
```
Q, R = @btime CGS_naive(A1);
Q, R = @btime CGS(A1);
@printf "||A - QR|| = %.2E, ||I - Q'Q|| = %.2E" norm(A1 - Q * R) norm(I - Q'Q)
```

```
  20.390 ms (6184 allocations: 158.73 MiB)
  6.163 ms (2806 allocations: 4.52 MiB)
||A - QR|| = 2.45E-15, ||I - Q'Q|| = 1.53E-05
```

[21]:
```
Q, R = @btime CGS(A2);
@printf "||A - QR|| = %.2E, ||I - Q'Q|| = %.2E" norm(A2 - Q * R) norm(I - Q'Q)
```

```
  6.172 ms (2806 allocations: 4.52 MiB)
||A - QR|| = 2.44E-15, ||I - Q'Q|| = 9.96E-04
```

[22]:
```
Q, R = @btime MGS_naive(A1);
Q, R = @btime MGS(A1);
@printf "||A - QR|| = %.2E, ||I - Q'Q|| = %.2E" norm(A1 - Q * R) norm(I - Q'Q)
```

```
  68.025 ms (360006 allocations: 470.73 MiB)
  34.784 ms (180315 allocations: 236.30 MiB)
```

```
||A - QR|| = 3.82E-15, ||I - Q'Q|| = 1.50E-06
```

[23]:
```
Q, R = @btime MGS(A2);
@printf "||A - QR|| = %.2E, ||I - Q'Q|| = %.2E" norm(A2 - Q * R) norm(I - Q'Q)
```

```
  33.852 ms (180315 allocations: 236.30 MiB)
||A - QR|| = 3.82E-15, ||I - Q'Q|| = 1.35E-04
```

[24]:
```
Q, R = @btime CGS2_naive(A1);
@printf "||A - QR|| = %.2E, ||I - Q'Q|| = %.2E" norm(A1 - Q * R) norm(I - Q'Q)
Q, R = @btime CGS2(A1);
@printf "||A - QR|| = %.2E, ||I - Q'Q|| = %.2E" norm(A1 - Q * R) norm(I - Q'Q)
```

```
  43.959 ms (10164 allocations: 313.88 MiB)
||A - QR|| = 3.19E-15, ||I - Q'Q|| = 6.59E-15  11.967 ms (4398 allocations: 6.39
MiB)
||A - QR|| = 3.20E-15, ||I - Q'Q|| = 6.52E-15
```

[25]:
```
Q, R = @btime CGS2(A2);
@printf "||A - QR|| = %.2E, ||I - Q'Q|| = %.2E" norm(A2 - Q * R) norm(I - Q'Q)
```

```
  12.000 ms (4398 allocations: 6.39 MiB)
||A - QR|| = 3.15E-15, ||I - Q'Q|| = 6.71E-15
```